Speech & Document Extraction

Take-home exercise | **Role:** AI/ML Engineer (Vision & Speech) | **Deadline:** 4 days

What this is

An AI service with two capabilities:

- **Transcribe audio** in Bengali and English
- **Extract structured data from photographed lab reports** in English

We are not testing how much you can build. We are testing *how* you build — the boundaries you draw, the edge cases you catch, which tests you write, and whether it runs on someone else's machine.

A smaller, clean, well-tested submission beats a larger sprawling one. If you run out of time, stop and write down what you did not get to. Disclosed gaps cost you far less than undisclosed ones.

Tools and models

You may use AI coding assistants. We use them daily and expect you to.

You may use open-source models or commercial APIs, self-hosted or called over the network. Neither scores higher than the other. Say in `DECISIONS.md` what you picked and why.

Test data — you source it

We do not supply data. Assembling a test set is part of the exercise.

1. **Collect your own audio clips and lab report images**, commit them to the repository under `testdata/`, and describe in your README where they came from and why you chose them.
2. **Write reference transcripts** for your audio clips so you can measure transcription accuracy.
3. **We will test your service on inputs you have never seen.** Choose your test data accordingly — clean, easy samples will not tell you what we are going to find.

Endpoint 1 — Transcription

```
POST /api/v1/transcribe
```

Multipart upload: an audio file, plus a `language` field accepting `bn`, `en`, or `auto`.

Returns the transcript, detected language, audio duration, and which provider produced it.

1. **Provider adapter pattern.** At least two adapters behind one interface: one real provider (self-hosted or API), and one **mock** adapter replaying recorded responses from disk with no network call and no model load. Which one runs is decided by configuration, not a code change.
2. **Input validation.** Reject unsupported formats and files over 25 MB with a structured error response, not a stack trace.
3. **Audio containing no speech.** Some inputs are silence or ambient noise. Decide what your endpoint returns for these and make sure it does so reliably.

Endpoint 2 — Lab report extraction

```
POST /api/v1/documents/extract
```

Multipart upload: a photograph or scan of an **English-language medical lab report**. Expect them to be photographed at an angle, in poor light, sometimes with part of the page cut off.

A lab report has a **header block** of metadata and a **table of test results**.

Return:

```
{
  "meta": {
    "patient_name": "...",
    "age": "...",
    "sex": "...",
    "report_date": "...",
    "lab_name": "...",
    "reference_no": "..."
  },
  "results": [
    {
      "test_name": "...",
      "value": ...,
      "unit": "...",
      "reference_range": "...",
      "flag": "...",
      "raw_line": "..."
    }
  ]
}
```

4. **Provider adapter pattern**, same rule as endpoint 1: one real OCR provider, one mock.
5. **Every result must carry** `raw_line` — the exact text OCR returned for that row, preserved verbatim. Never dropped, never cleaned up.
6. **Every result must include a numeric** `value`.
7. **Value and unit normalisation.** Values appear as `12.5`, `<0.5`, `12,500`, `1.2 x 10^3`, `0.8 - 1.2`. Units appear as `mg/dL`, `gm/dl`, `g/dL`, `mmol/L`, `10^3/µL`. Dates appear in several formats.

   Normalise to a canonical form of your own design. Document it in your README.

   Anything you cannot confidently parse must be preserved verbatim, never guessed at.
8. **Handle documents that are not lab reports** without producing garbage. Degrade gracefully.

Engineering requirements

9. **Python 3.11+, FastAPI.** Type hints throughout.
10. **Layer separation.** Three layers, dependencies pointing inward:

    ```
    api/        HTTP routing, request/response models, validation
    services/   orchestration and business logic
    adapters/   provider and model integration
    ```

    - No provider SDK or model library imported outside `adapters/`
    - No FastAPI type (`Request`, `Response`, `UploadFile`, `HTTPException`) in `services/`

    We check this mechanically.
11. **Docker.** `docker compose up` from a clean clone must bring the service up on the mock adapters, with **no credentials and no model download required**. Real adapters activate via `.env`.

    If you self-host a model, keep it off the default compose path — a reviewer must be able to run your service seamlessly.
12. **Tests.** Targeted, not exhaustive. Meaningful tests on the value normaliser, the validation path, and one integration test per endpoint against the mocks. Fifteen sharp tests beat a hundred asserting HTTP 200.

13. **Configuration** via environment variables with a typed settings object. **No secrets in the repository** — this is going to be public, so check twice.
14. **Git.** Micro-commits with clear messages. We read your log as a document — it is the most informative artifact you will submit. Do not squash. Do not tidy the history afterward; we would rather see the messy truth.

Documentation

15. `README.md` — how to run it, your architecture and why, your normalised value format, where your test data came from and why you chose it, and known limitations.
16. `DECISIONS.md` — three to five short entries on consequential choices, including your model choice. What you picked, what you rejected, why. One line each is too thin; a page each is too much.

How we grade this

So you know where to spend your time. Weights are out of 100.

Code quality & tests	**20**
Architecture & layer separation	**18**
Git history	**18**
Correctness & edge-case handling	**16**
Test data, and how you chose it	**12**
Runs cleanly — Docker, config, validation	**10**
Documentation & judgment	**6**

What earns points

- Small commits that each do one thing, with messages that say *why*
- `docker compose up` working on a clean clone, first try, with no credentials
- Tests that would actually fail if the code broke
- Test data chosen to break your own service rather than to make it look good
- Telling us you did not get to something, instead of hiding it
- Telling us a requirement in this brief is ambiguous, contradictory or wrong
- A `DECISIONS.md` entry that names what you rejected and why, not just what you chose

What does not

- Volume — extra endpoints, extra features, extra files
- A polished README wrapped around code that does not run
- High test coverage made of assertions on status codes
- Infrastructure we did not ask for — queues, caches, orchestration
- Guessing a value you could not parse. We would rather see it preserved verbatim
- Squashed or rewritten history. Secrets committed, even revoked ones

Automatic no

> We will ask you to walk us through your code. If you cannot explain how a non-trivial part of your own submission works, or the repository turns out to be copied from someone else's, that ends the application regardless of everything above.

Submission

Push to a public GitHub repository under your own account and send us the link.

Full history, unsquashed. Do not create the repo on day three and push everything at once — we read the commit history, and an empty one tells us as much as a bad one.

If anything here is unclear or looks wrong, email us. Asking is not a penalty.